Prüfung I - Modul 295 (Musterlösung)

Backend für Applikationen realisieren

Dienstag, 27. Oktober 2026

Name:

Klasse:

Punkte (von 71):

Aufgabe	A1	A2	A3	A4	A5	A6	A7	Total
Max.	15	8	9	7	17	10	5	71
Erreicht								

Note:

Hinweise

- Hilfsmittel: keine. Schreiben Sie Ihre Antworten direkt auf die Prüfungsblätter.
- Die Aufgaben sind unabhängig voneinander und können in beliebiger Reihenfolge gelöst werden.
- Antworten Sie kurz und präzise. Stichworte genügen, sofern die Aussage klar ist.
- Bei Code, den Sie selbst schreiben, gibt es für kleine Syntaxfehler (z.B. ein fehlendes Semikolon) keinen Abzug.

1 Grundlagen, Datentypen und Operatoren (15p)

1.1 PHP wird *serverseitig* ausgeführt. Was bedeutet das, und was erhält der Browser vom Server zurück? **(2p)**

Lösung: Der PHP-Code wird auf dem Webserver ausgeführt, nicht im Browser. Der Browser erhält nur das Ergebnis, also fertiges HTML (ggf. mit CSS/JS). Den PHP-Quellcode sieht er nie. (1p Ausführung auf Server, 1p Browser erhält HTML)

1.2 Wofür wird XAMPP benötigt? In welchem Ordner müssen die PHP-Dateien liegen, und wie ruft man ein Projekt `shop` danach im Browser auf? **(2p)**

Lösung: XAMPP liefert einen lokalen Webserver (Apache) mit PHP (und MySQL/MariaDB), damit PHP-Dateien ausgeführt werden können (1p). Dateien liegen im Ordner `htdocs`, Aufruf über `http://localhost/shop/` (1p).

1.3 Nennen Sie die vier skalaren Datentypen von PHP mit je einem Beispielwert. **(2p)**

Lösung: `string` (`"Hallo"`), `int` (`24`), `float` (`19.95`), `bool` (`true`). Je ½p.

1.4 Erklären Sie den Unterschied zwischen *Type Juggling* und *Type Casting*. Geben Sie zu beidem ein kurzes Beispiel. **(2p)**

Lösung: Type Juggling: PHP wandelt Typen bei Bedarf *automatisch* um, z.B. `"10"+ 5` ergibt `int(15)` (1p). Type Casting: Man wandelt einen Wert *gezielt selbst* um, z.B. `(int) "42"` (1p).

1.5 Ergibt der Wert bei der Umwandlung in `bool` den Wert `true` oder `false`? Kreuzen Sie an. **(3p)**

Wert	true	false
`"0"`		X
`"false"`	X	
(ein Leerzeichen)	X	
`[]`		X
`-1`	X	
`null`		X

Je ½p.

1.6 Was ist der Unterschied zwischen `==` und `===`? Welchen der beiden Operatoren sollte man in der Regel verwenden, und weshalb? **(2p)**

Lösung: `==` vergleicht nur den Wert (mit Type Juggling), `===` vergleicht Wert *und* Typ (1p). Man sollte `===` verwenden, um unerwartete Ergebnisse durch Type Juggling zu vermeiden (1p).

1.7 Was ist eine Konstante, und wie unterscheidet sie sich von einer Variable? Nennen Sie zwei Arten, eine Konstante zu definieren. **(2p)**

Lösung: Eine Konstante speichert einen Wert, der sich zur Laufzeit nicht mehr ändern lässt; sie hat kein `$` vor dem Namen (1p). Definition mit `define("VAT", 0.077);` oder `const VAT = 0.077;` (1p).

2 Arrays, Schleifen und Funktionen (8p)

2.1 Was ist der Unterschied zwischen einem indizierten und einem assoziativen Array? Schreiben Sie zu beiden ein kurzes Beispiel. **(2p)**

Lösung: Indiziert: PHP vergibt die Schlüssel automatisch ab 0, z.B. `[Äpfel", "Birne"]` (1p). Assoziativ: Die Schlüssel werden selbst definiert (meist Strings), z.B. `["name-> Änna", äge-> 23]` (1p).

2.2 Was ist der Unterschied zwischen einer `while`- und einer `do-while`-Schleife? **(2p)**

Lösung: Bei `while` wird die Bedingung *vor* jedem Durchlauf geprüft; ist sie von Anfang an `false`, läuft die Schleife nie (1p). Bei `do-while` wird sie erst *nach* dem Durchlauf geprüft, die Schleife läuft also mindestens einmal (1p).

2.3 Was ist der Unterschied, ob eine Funktion ein Ergebnis mit `echo` ausgibt oder mit `return` zurückgibt? **(2p)**

Lösung: `echo` schreibt den Wert nur in die Ausgabe, danach steht er im Programm nicht mehr zur Verfügung (1p). `return` gibt den Wert an die Aufrufstelle zurück, wo er weiterverwendet werden kann (z.B. in einer Variable); zudem beendet `return` die Funktion (1p).

2.4 Was versteht man unter dem *Gültigkeitsbereich* (Scope) einer Variable in einer Funktion? Wie gelangt ein Wert von ausserhalb am besten in eine Funktion hinein? **(2p)**

Lösung: Variablen innerhalb einer Funktion sind lokal und existieren nur dort; Variablen von ausserhalb sind in der Funktion nicht sichtbar (1p). Am besten übergibt man den Wert als Parameter (und gibt das Ergebnis mit `return` zurück); `global` ist möglich, aber zu vermeiden (1p).

3 Klassen (9p)

3.1 Erklären Sie den Unterschied zwischen einer Klasse und einem Objekt. **(2p)**

Lösung: Eine Klasse ist ein Bauplan, der festlegt, welche Eigenschaften und Methoden ein Objekt hat (1p). Ein Objekt ist eine konkrete Instanz, die mit `new` nach diesem Bauplan erzeugt wird; es gibt beliebig viele, unabhängige Objekte (1p).

3.2 Von wo ist eine Eigenschaft mit der jeweiligen Sichtbarkeit zugreifbar? Kreuzen Sie alle zutreffenden Felder an. **(3p)**

	eigene Klasse	**abgeleitete Klasse**	**von aussen**
`public`	X	X	X
`protected`	X	X	
`private`	X		

1p pro korrekte Zeile.

3.3 Wozu verwendet man Getter und Setter? Welchen Vorteil hat ein Setter gegenüber einer `public`-Eigenschaft? **(2p)**

Lösung: Getter/Setter sind öffentliche Methoden, über die man `private`-Eigenschaften lesen bzw. ändern kann (Kapselung) (1p). Ein Setter kann den Wert vor dem Übernehmen prüfen und ungültige Werte (z.B. negativen Preis) ablehnen (1p).

3.4 Was ist der Unterschied zwischen einer abstrakten Klasse und einem Interface? **(2p)**

Lösung: Eine abstrakte Klasse kann nicht mit `new` erzeugt werden, kann aber neben abstrakten Methoden auch fertige Methoden (und Eigenschaften) enthalten, die vererbt werden; man erbt mit `extends` (1p). Ein Interface schreibt nur vor, welche Methoden eine Klasse haben muss, ohne Umsetzung; es wird mit `implements` umgesetzt, und eine Klasse kann zusätzlich von einer Klasse erben (1p).

4 Sessions (7p)

4.1 HTTP ist *zustandslos*. Was bedeutet das, und welches Problem entsteht dadurch z.B. bei einem Login? **(2p)**

Lösung: Jeder Request ist unabhängig, der Server erinnert sich nicht an frühere Requests; Variablen existieren nur während eines Requests (1p). Ohne Sessions wäre z.B. der Login-Status beim nächsten Seitenaufruf wieder weg (1p).

4.2 Erklären Sie, wie Session-ID, Cookie und Server zusammenspielen, damit der Server einen Besucher wiedererkennt. Wo liegen die Session-Daten? **(3p)**

Lösung: `session_start()` erzeugt beim ersten Besuch eine eindeutige Session-ID und schickt sie als Cookie (`PHPSESSID`) an den Browser (1p). Der Browser sendet das Cookie bei jedem weiteren Request automatisch mit, und PHP findet anhand der ID die passenden Daten wieder (1p). Die Daten selbst liegen auf dem Server, im Cookie steht nur die ID (1p).

4.3 Worin unterscheiden sich `session_destroy()` und `unset($_SESSION["cart"])`? **(2p)**

Lösung: `session_destroy()` löscht alle Daten der Session auf dem Server, z.B. beim Logout (1p). `unset($_SESSION["cart"])` entfernt nur diesen einen Wert, alle anderen Session-Daten bleiben erhalten (1p).

5 Code verstehen (17p)

Was gibt der Code jeweils aus? Alle Code-Ausschnitte sind vollständig und funktionieren unabhängig voneinander.

5.1 (2p)

```php
<?php
var_dump("10" + 4);
var_dump("10" . 4);
var_dump("10" == 10);
var_dump("10" === 10);
```

Lösung: `int(14)`, `string(3) "104"`, `bool(true)`, `bool(false)`. Je ½p (Typ muss stimmen).

5.2 (2p)

```php
<?php
$x = 4;
$x *= 2 + 1;
$x %= 5;
echo $x ** 3;
```

Lösung: 8. (`$x = 4 * 3 = 12`, `12 % 5 = 2`, `2 ** 3 = 8`; 1p Zwischenschritt `12` bzw. `2`)

5.3 (2p)

```php
<?php
$level = "silber";
$discount = 0;
switch ($level) {
    case "gold":
        $discount += 20;
        break;
    case "silber":
        $discount += 10;
    case "bronze":
        $discount += 5;
        break;
    default:
        $discount = 0;
}
echo $discount;
```

Lösung: `15`. Bei `ßilber"` fehlt das `break`, die Ausführung fällt in `"bronze"` (Fallthrough). 2p nur für `15`.

5.4 (2p)

```php
<?php
$sum = 0;
for ($i = 1; $i <= 10; $i++) {
    if ($i % 3 == 0) {
        continue;
    }
    if ($sum > 10) {
        break;
    }
    $sum += $i;
}
echo $sum;
```

Lösung: 12. (1 + 2 + 4 + 5; 3 und 6 werden übersprungen, bei `$i = 7` ist `$sum > 10` und die Schleife bricht ab)

5.5 (2p)

```php
<?php
$prices = ["Apfel" => 0.5, "Brot" => 3.2, "Käse" => 7.9];
$prices["Milch"] = 1.6;
unset($prices["Apfel"]);
$expensive = array_filter($prices, fn($p) => $p > 2);
echo count($prices) . " " . count($expensive);
```

Lösung: `3 2`. (Brot, Käse, Milch; davon über 2: Brot, Käse. Je 1p)

5.6 (2p)

```php
<?php
$total = 10;

function setTotal($n) {
    $total = $n;
    return $total;
}

function addFive(&$n) {
    $n += 5;
}

setTotal(99);
echo $total . " ";
addFive($total);
echo $total;
```

Lösung: `10 15`. `setTotal()` ändert nur eine lokale Variable (Scope), `addFive()` erhält eine Referenz und ändert das Original. Je 1p.

5.7 (3p)

```php
<?php
class Ticket {
    private static int $next = 1;
    public int $nr;

    public function __construct(public string $type = "Normal") {
        $this->nr = self::$next++;
    }

    public function label(): string {
        return $this->type . " #" . $this->nr;
    }
}

class VipTicket extends Ticket {
    public function label(): string {
        return "VIP " . parent::label();
    }
}

$a = new Ticket();
$b = new VipTicket("Gold");
echo $a->label() . " / " . $b->label();
```

Lösung: `Normal #1 / VIP Gold #2`. 1p `Normal #1` (Default-Wert), 1p `#2` (statischer Zähler wird von allen Objekten geteilt, auch von der Kindklasse), 1p `VIP Gold` (überschriebene Methode mit `parent::`).

5.8 Die folgende Seite wird im selben Browser viermal nacheinander aufgerufen. Was wird bei jedem Aufruf ausgegeben? **(2p)**

```php
<?php
session_start();
$_SESSION["visits"] = ($_SESSION["visits"] ?? 0) + 1;

if ($_SESSION["visits"] >= 3) {
    echo "Stammgast";
    unset($_SESSION["visits"]);
} else {
    echo $_SESSION["visits"];
}
```

Lösung: 1. Aufruf: `1`, 2. Aufruf: `2`, 3. Aufruf: `Stammgast`, 4. Aufruf: `1` (`unset()` hat den Zähler entfernt). Je ½p.

6 Fehler finden (10p)

Jeder der folgenden Code-Ausschnitte enthält einen Fehler: Er funktioniert nicht oder macht nicht das, was der Kommentar verlangt. Beschreiben Sie den Fehler und wie man ihn behebt. (je 2p: 1p Fehler, 1p Korrektur)

6.1

```php
<?php
$name = "Anna";
echo 'Hallo $name!'; // soll "Hallo Anna!" ausgeben
```

Lösung: In einfachen Anführungszeichen werden Variablen nicht ersetzt, ausgegeben wird `Hallo $name!`. Korrektur: doppelte Anführungszeichen `"Hallo $name!"` oder Konkatenation `'Hallo ' . $name . '!'`.

6.2

```php
<?php
function greet($greeting = "Hallo", $name) {
    return $greeting . " " . $name;
}
echo greet("Anna"); // soll "Hallo Anna" ausgeben
```

Lösung: Der optionale Parameter steht vor dem Pflichtparameter. `Änna"` wird `$greeting` zugeordnet, für `$name` fehlt ein Argument (Fehler). Korrektur: `function greet($name, $greeting = "Hallo")`.

6.3

```php
<?php
// soll die Zahlen 1 bis 5 ausgeben
$i = 1;
while ($i <= 5) {
    echo $i;
}
```

Lösung: `$i` wird nie erhöht, die Bedingung bleibt immer `true`: Endlosschleife. Korrektur: `$i++;` in der Schleife ergänzen.

6.4

```php
<?php
class Product {
    public function __construct(
        public string $name,
        private float $price
    ) {}
}

$apple = new Product("Apfel", 0.5);
echo $apple->price; // soll den Preis ausgeben
```

Lösung: `$price` ist `private` und von aussen nicht zugreifbar (Fehler). Korrektur: Getter `getPrice(): float { return $this->price; }` ergänzen und `$apple->getPrice()` aufrufen (oder `public` verwenden).

6.5

```php
<?php
echo "<h1>Willkommen</h1>";
session_start();
$_SESSION["username"] = "Anna"; // soll gespeichert werden
```

Lösung: `session_start()` wird erst nach einer Ausgabe aufgerufen; es muss aber vor jeder Ausgabe stehen (sonst kann das Session-Cookie nicht mehr gesendet werden). Korrektur: `session_start()` ganz an den Anfang des Skripts.

7 Code schreiben (5p)

7.1 Schreiben Sie die beiden Codeblöcke jeweils als eine einzige Zeile: den ersten mit dem Null-Coalescing-Operator `??`, den zweiten mit dem ternären Operator `?:`. **(2p)**

```php
<?php
if (isset($_GET["lang"])) {
    $lang = $_GET["lang"];
} else {
    $lang = "de";
}

if ($age >= 18) {
    $status = "volljährig";
} else {
    $status = "minderjährig";
}
```

```php
$lang = $_GET["lang"] ?? "de";
$status = $age >= 18 ? "volljährig" : "minderjährig";
```

Je 1p.

7.2 Schreiben Sie eine Funktion `countPassed(array $grades): int`, welche die Anzahl genügender Noten (Note `>= 4`) im Array zurückgibt. **(3p)**

Beispiel: `countPassed([4.5, 3.5, 6, 4])` gibt `3` zurück.

```php
function countPassed(array $grades): int {
    $count = 0;
    foreach ($grades as $grade) {
        if ($grade >= 4) {
            $count++;
        }
    }
    return $count;
}
```

1p Schleife über das Array, 1p Bedingung und Zähler, 1p `return`. Auch `count(array_filter($grades, fn($g) => $g >= 4))` ist korrekt.